

Software Requirements Specification

Team - 3 Hostel Management System

Team Members

Name	SRN
Shalik S	PES2UG24CS454
Shreyas Patil	PES2UG24CS492
Shreyas Mohanty	PES2UG24CS491
Shobhit Rachit Keshri	PES2UG24CS475

1. Introduction

1.1 Purpose

Hostel Management System (HMS) is a web application that helps ease student registration, room setup and allocation, fee tracking, complaints, and visitor logging. This document is to keep track of what we are building and is to be used as reference whenever needed.

1.2 Intended Audience

- The team - Sections 3-6 are what we're implementing.
- The instructor - Sections 1-2 for scope, Section 5 for the requirements themselves.
- Others - the whole document, glossary in Appendix A first.

Read alongside the shorter Functional and Non-Functional Requirements document, which lists the same requirements in table form.

1.3 Product Scope

Right now our hostel tracks room allotment and fees on paper registers and spreadsheets. That makes it slow to check which rooms are free, making it easy to accidentally put two students in the same room and hard to keep track of complaints that never got followed up on.

HMS gives students a portal to apply for a room, check their fee status, raise complaints and log visitors. It gives the warden one dashboard instead of a stack of registers. The main goals are to stop double-allotment, show occupancy in real time and give every complaint a status that doesn't get lost.

1.4 References

1. Project Proposal
2. Functional and Non-Functional Requirements

2. Overall Description

2.1 Product Perspective

HMS is a web application meant to ease the life of the warden and the students.

2.2 Product Functions

- Student registration and role-based login
- Block/room setup by Admin (capacity, type, fee)
- Room application by students and allocation, reassignment and deallocation by Admin
- Live occupancy view by block/floor
- Fee recording, balance calculation, receipts
- Complaint submission and status tracking
- Visitor entry/exit logging
- Admin dashboard (occupancy, dues, open complaints)
- Search/filter across students, rooms, complaints

2.3 Users

User Class	What they do	How often
Student	Registers, applies for a room, checks fee status, raises complaints, logs visitors	A few times a month
Admin (Warden)	Sets up rooms, allots them, records fees, resolves complaints, pulls reports	Daily

A student can only see and edit their own stuff. Admin can see and edit everything.

2.4 Operating Environment

Backend: Python 3.10+, Django

Database: SQLite or PostgreSQL

Frontend: Django templates, server-rendered

Client: current Chrome or Firefox

Deployment: one server

Constraints

- Login and roles use Django's built-in auth
- Room numbers must be unique within a block, enforced by the database
- Fee payment is recorded, not processed. No real payment gateway.

2.6 Assumptions

- One hostel, one college. Not multi-tenant.
- Students pay fees at the college office; Admin records it afterward.
- Blocks and rooms are set up by Admin before the allotment period starts.
- We're relying on Django's standard packages (auth, ORM, admin site).

3. External Interface Requirements

3.1 User Interfaces

Every page has the same header/nav depending on the logged-in role, with a visible logout button. Form errors show next to the field that's wrong and don't clear what was already typed.

Screen	Who sees it	What's on it
Login	Everyone	Username, password, link to register
Registration	New students	SRN, name, email, phone, password
Student dashboard	Student	Room status, fee balance, own complaints, visitor log form
Room application	Student	Preferred block/type, current application status
Admin dashboard	Admin	Occupancy %, dues, open complaint count, pending applications
Room management	Admin	Add/edit blocks and rooms, room list with occupancy
Allocation	Admin	Pending applications, vacant rooms, allot/deallocate actions

Screen	Who sees it	What's on it
Fee management	Admin	Fee list per student, record-payment form, receipts
Complaint queue	Admin	Complaints by status/category, status update
Reports	Admin	Occupancy, fee-due, and complaint reports, CSV download

3.2 Hardware Interfaces

None, it runs on ordinary server and client hardware.

3.3 Software Interfaces

- Django ORM to PostgreSQL/SQLite for everything.
- Django admin site available to a superuser for fixing data directly if needed
- CSV export using Python's built-in csv module.

3.4 Communications Interfaces

- HTTPS in deployment.
- CSRF protection stays on for every form that changes data.
- No outbound API calls, status changes just show in the app.

4. Analysis Models

Actors: Student, Admin (Warden).

Use cases: Register, Log In, Apply for Room, Add Block/Room, Allot Room, Deallocate Room, View Availability, Record Payment, Generate Receipt, Raise Complaint, Update Complaint Status, Log Visitor, Record Checkout, View Dashboard, Generate Report, Search.

5. System Features

5.1 Login and Roles

Priority: High — everything else sits behind this.

A user submits credentials, the system checks them, and sends the user to the right dashboard. Wrong credentials generic error.

FR-1.1 Authenticate against stored username/password before allowing access to anything but login/registration.

FR-1.2 Invalid credentials get one generic error

FR-1.3 Redirect to Student or Admin dashboard based on the account's role.

FR-1.4 Block Student accounts from Admin-only routes, even by direct URL.

FR-1.5 Logout ends the session, no page should still work after that without logging in again.

5.2 Student Registration

Priority: High.

A prospective student fills the registration form and gets a Student account. Admin can view/edit any student afterward.

FR-2.1 Register with SRN, name, email, phone, and password.

FR-2.2 Reject registration if the SRN or email already exists.

FR-2.3 A student can edit their own contact info, not their SRN or role.

FR-2.4 Admin can view and edit any student's full record, including branch and year.

5.3 Block and Room Setup

Priority: High

FR-3.1 Admin creates a block (name, number of floors).

FR-3.2 Admin creates a room under a block: number, floor, capacity, type (single/double/triple), fee.

FR-3.3 Room numbers are unique within a block — enforced at the DB level.

FR-3.4 Admin can edit a room's capacity, type, and fee.

FR-3.5 A block or room with a current occupant can't be deleted.

5.4 Room Allocation

Priority: High

A student applies, Admin sees the pending application, picks a vacant room, and allots it. The student sees it on their dashboard right after.

- FR-4.1 A student can apply for a room, optionally stating a preferred block/type.
- FR-4.2 Admin sees pending applications with student name, SRN, and preference.
- FR-4.3 Every room shows its status and occupancy count.
- FR-4.4 A room already at capacity can't be allotted further.
- FR-4.5 A student who already holds a room can't be allotted a second one.
- FR-4.6 Allotment updates the room count and student record together
- FR-4.7 Admin can deallocate or reassign a student.
- FR-4.8 The student's dashboard shows the new allotment right after Admin confirms it.

5.5 Fee Management

Priority: Medium.

- FR-5.1 Store fee due and fee paid per student per term. Fee due comes from the room's assigned fee.
- FR-5.2 Admin records a payment (amount + date).
- FR-5.3 Balance is calculated automatically (due minus paid), not typed in by Admin.
- FR-5.4 A payment bigger than the outstanding balance is rejected.
- FR-5.5 A receipt is generated per payment: student, SRN, room, amount, date, balance.
- FR-5.6 A student can view their own fee status and download their own receipts only.

5.6 Complaint Management

Priority: Medium.

- FR-6.1 Submit a complaint with category and description.
- FR-6.2 The complaint auto-tags the student's room and the time it was submitted.
- FR-6.3 New complaints start as Open.
- FR-6.4 Admin moves it through Open In Progress Resolved
- FR-6.5 A student can't change the status of any complaint, including their own.
- FR-6.6 A student sees only their own complaints, Admin sees all of them.

5.7 Visitor Log

Priority: Low.

- FR-7.1 Record visitor name, phone, purpose, host student, entry time.
- FR-7.2 Entry time comes from the server clock, not typed by the user.
- FR-7.3 Admin marks a visitor as checked out, recording exit time.
- FR-7.4 Admin can list visitors currently on premises (entry logged, no exit yet).
- FR-7.5 A visitor entry needs a host student who currently holds a room.

5.8 Admin Dashboard

Priority: Medium

- FR-8.1 Show total rooms, occupied rooms, occupancy %.
- FR-8.2 Show count of students with dues and the total amount owed.
- FR-8.3 Show count of Open/In Progress complaints.
- FR-8.4 Show count of pending room applications.
- FR-8.5 Every number is pulled live from the database, not cached.

5.9 Search and Filter

Priority: Medium

- FR-9.1 Admin searches students by name or SRN (partial match).
- FR-9.2 Admin filters rooms by block, floor, type, status.
- FR-9.3 Admin filters complaints by status and category.
- FR-9.4 No matches “no results found” message, not an error.
- FR-9.5 A student’s search only covers their own records.

6. Other Non-Functional Requirements

6.1 Performance

Room list, dashboard, and search pages load within 2 seconds under normal use.

6.2 Safety

Database backup so that no data is lost.

6.3 Security

Passwords are hashed with Django’s default (PBKDF2), never stored as plain text. Every Admin route checks the role server-side, not just by hiding a link. A student can’t view another student’s record by changing an ID in the URL. HTTPS in deployment, CSRF stays on for every data-changing form.

6.4 Quality Attributes

Reliability. Two Admins allotting the last free bed at the same time one succeeds, one is rejected, occupancy never exceeds capacity. Multi-step actions (like allotment) either fully complete or don’t happen at all.

Usability. Applying for a room and raising a complaint are within two clicks of the student dashboard; recording a payment is within two clicks of the Admin dashboard.

Maintainability. Code split into separate Django apps (accounts, rooms, fees, complaints, visitors), each with its own basic tests.

Scalability. Should handle roughly 1000 students and 500 rooms without a schema rewrite.

Portability. Runs on Windows/Linux/macOS with Python 3.10+, installed from one requirements.txt. Works on current Chrome and Firefox.

Availability. Up during hostel office hours (9 am–6 pm, working days) with minimal downtime. A crash shows a plain error page, not a stack trace.

6.5 Business Rules

- Only Admin can allot/deallocate rooms, mark fees paid, or resolve complaints.
- A student can hold at most one active room allocation.
- A room's occupancy can never exceed its capacity.
- Only Admin can mark a complaint Resolved.
- A block or room with a current occupant can't be deleted.

7. Other Requirements

Database structure follows the Django models in Appendix B - migrations are kept in the repo so the schema can be rebuilt from scratch. No legal/regulatory requirements apply. Interface is English only.

Appendix A: Glossary

Term Meaning

HMS: Hostel Management System

SRN: Student Registration Number (the college's student ID)

Admin: The privileged role that is held by the warden or hostel office staff.

Warden: The person holding the Admin account. Same as Admin here.

Allocation: Assigning a room to a student for a term

Occupancy: How many students are currently in a room

Appendix B: Field Layouts

Student

Field	Length	Type	Required
SRN	13	Alphanumeric, unique	Yes
Name	60	String	Yes
Email	100	String, unique	Yes
Contact number	15	String	Yes
Branch	30	String	No
Year of study	1	Numeric	No
Password (hashed)	128	String	Yes
Role	10	String (Student/Admin)	Yes

Block

Field	Length	Type	Required
Block name	20	String, unique	Yes
Number of floors	2	Numeric	Yes

Room

Field	Length	Type	Required
Room number	6	Alphanumeric, unique in block	Yes
Block	20	String (FK)	Yes
Floor	2	Numeric	Yes
Capacity	1	Numeric (1–3)	Yes
Room type	10	String (Single/Double/Triple)	Yes
Fee per term	10	Numeric	Yes
Current occupancy	1	Numeric	Yes

Allocation

Field	Length	Type	Required
Student SRN	13	Alphanumeric (FK)	Yes
Room number	6	Alphanumeric (FK)	Yes
Allotment date	8	Date	Yes
Vacated date	8	Date	No
Active	1	Boolean	Yes

Fee

Field	Length	Type	Required
Student SRN	13	Alphanumeric (FK)	Yes
Term	10	String	Yes
Amount due	10	Numeric	Yes
Amount paid	10	Numeric	Yes
Last payment date	8	Date	No
Receipt number	12	Alphanumeric, auto	No

Complaint

Field	Length	Type	Required
Complaint ID	10	Numeric, auto	Yes
Student SRN	13	Alphanumeric (FK)	Yes
Room number	6	Alphanumeric	Yes
Category	30	String	Yes
Description	500	String	Yes
Status	15	String (Open/In Progress/Resolved)	Yes
Raised at	—	DateTime	Yes

Visitor

Field	Length	Type	Required
Visitor name	60	String	Yes
Contact number	15	String	Yes
Purpose	100	String	Yes
Host SRN	13	Alphanumeric (FK)	Yes
Entry time	—	DateTime	Yes
Exit time	—	DateTime	No

Appendix C: Test Case Traceability

Requirement #	Test Case	Result/Description
1	FR-1.2	Wrong password generic error, no hint which field was wrong
2	FR-1.4	Student requesting an Admin URL directly blocked
3	FR-2.2	Registering with an SRN already in use rejected
4	FR-3.3	Two rooms, same number, same block rejected by the database
5	FR-4.4	Allotting a full room rejected
6	FR-4.5	Allotting a second room to an already-allotted student rejected
7	FR-5.4	Payment bigger than the balance owed rejected
8	FR-6.3	New complaint status is Open
9	FR-6.5	Student tries to resolve their own complaint rejected
10	FR-7.5	Visitor logged against a student with no room rejected
11	FR-9.4	Search with no matches empty list with a message, not an error
12	NFR-3	Student changes URL ID to view another student's record blocked
13	NFR-4	Two simultaneous allotments for the last bed one wins, one rejected